

# Variable Selection using Artificial Neural Networks

Maxime Van Cutsem

MAXIME.VANCUTSEM@UNIGE.CH

Editor: XXX

## Abstract

Variable selection in neural networks remains challenging due to their nonlinear and highly parameterized structure, in particular because first-layer weights cannot be interpreted independently of the downstream network. Yet, selecting a relevant subset of input variables can improve interpretability while reducing model size. We introduce an adaptive first-layer regularization framework for multilayer perceptrons relying on a single regularization parameter. Each first-layer neuron is weighted by its downstream sensitivity, yielding a scale-invariant penalty and normalizing the corresponding first-layer gradients. The penalized problem is solved on the representatives of the rescaling symmetry where every sensitivity equals one, where it becomes a lasso on the first layer, by a proximal adaptive gradient method whose zero-thresholding test is the calibrated one. To preserve variables whose relevance emerges only through nonlinear interactions, the regularization level is raised progressively along a warm-started path. The resulting sparse networks discard irrelevant input variables and their associated parameters, thereby reducing both input dimensionality and model complexity. We develop the framework for Gaussian regression, binary classification, and Cox proportional-hazards models, and investigate its variable-selection and sparsification properties through numerical experiments.

**Keywords:** neural networks, variable selection, sparse neural networks, adaptive regularization, pivotal loss; proximal optimization

## 1 Introduction

In machine learning, models ranging from linear methods to artificial neural networks (ANNs) have achieved remarkable success: the former owing largely to their simplicity and interpretability, and the latter to their flexibility and ability to learn complex relationships from data. Selecting which features a model should be allowed to use, however, is a statistical problem before it is a computational one. In the spirit of Occam’s razor—*“Entities must not be multiplied beyond necessity”*—among models with comparable predictive performance, preference should be given to those that achieve it with fewer parameters. Yet modern machine learning has increasingly favoured predictive performance and large-scale parametrization over parsimony. As computational power and storage have become more readily available, it has become feasible to train increasingly large models using many inputs and parameters, often without explicitly questioning whether all of them are necessary. In this regime, overparameterization is computationally affordable, but computational feasibility does not imply statistical necessity. A neural network may therefore achieve strong predictive performance while relying on an unnecessarily large set of inputs and parameters, making it difficult to distinguish the variables that genuinely contribute to the model from those that are merely accommodated by its flexibility. As a result, the input-layer weight

matrix  $W_1$  may remain unnecessarily dense: irrelevant input variables retain connections to the hidden layer even though these connections are not needed for prediction. Identifying and removing such variables therefore not only improves interpretability but can also substantially reduce the number of parameters required by the network.

For linear models, this variable-selection problem is well posed and extensively studied. The lasso of Tibshirani (1996), for instance, produces a family of supports indexed by a regularization parameter  $\lambda$ , and a substantial literature addresses how this parameter should be chosen. In neural networks, the same question arises in a fundamentally different setting: the parameterization is non-identifiable, the training objective is non-convex, and the fitted model is typically obtained through first-order iterative optimization rather than by solving the optimization problem to global optimality. Sparsity-inducing penalties on the input layer extend the principle of variable selection to this setting (Feng and Simon, 2019; Lemhadri et al., 2021), but the crucial choice of  $\lambda$  is still most commonly delegated to cross-validation.

Cross-validation is unattractive for variable selection for two reasons. First, it is computationally expensive, as it multiplies the cost of an already demanding training procedure by the number of folds and candidate values of  $\lambda$ . More fundamentally, it targets predictive risk rather than variable-selection accuracy: the value of  $\lambda$  that minimizes prediction error need not recover the underlying support and may retain variables carrying little or no signal. If the objective is variable selection, the regularization level should instead be calibrated directly against the event one seeks to control—the inclusion of variables carrying no predictive signal. A natural approach is therefore to calibrate  $\lambda$  under the pure-noise model, in which no variable carries predictive signal, so that, for a prescribed  $\alpha$ , no variable is selected with probability at least  $1 - \alpha$ . Such null-based calibration has been developed for Gaussian linear regression (Donoho and Johnstone, 1994; Belloni et al., 2011) and, more recently, extended to a broader class of loss functions through the pivotal information criterion of Sardy et al. (2026). The pivotal construction makes the null distribution of the calibrating statistic independent of unknown nuisance parameters, allowing  $\lambda$  to be calibrated without estimating quantities such as the noise scale. In this work, we adopt this pivotal framework and the corresponding loss functions considered in Sardy et al. (2026).

Transferring this calibration principle to neural networks introduces an additional difficulty. The gradient with respect to the input-layer weights depends on the downstream network weights, which, because neural-network parameterizations are non-identifiable, can be rescaled without changing the function represented by the network while simultaneously changing the magnitude of the input-layer gradients. Consequently, a regularization threshold based directly on these gradients is itself parameterization-dependent: the same fitted function may cross or remain below the threshold solely as a consequence of rescaling. A meaningful null calibration for neural networks must therefore account for the arbitrary scale induced by the network parameterization.

This paper addresses this difficulty by exploiting the structure of the input-layer gradient under the null. We show that, when the input layer is empty, this gradient is exactly a rank-one matrix: the outer product of a network-sensitivity vector and a data-dependent vector. The data-dependent factor carries the evidence against the null and is calibrated once, before training, by Monte Carlo, whereas the sensitivity factor captures the current scale of the network and is recomputed at every iteration. This decomposition yields an

adaptive regularization level that follows the scale of the network throughout training, making variable selection invariant to equivalent rescalings of its parameterization while preserving the original null calibration. Following the pivotal framework described above, we focus on three tasks: regression, binary classification, and Cox survival analysis.

### Contributions.

- A zero-thresholding condition for a penalized multilayer perceptron (Theorem 4) and an exact rank-one factorization of the null gradient (proof of Theorem 4) separating the statistical evidence from the network amplification.
- A regularization rule,  $\lambda^{(t)} = \lambda_{\alpha}^{\text{DB}} s(\boldsymbol{\theta}^{(t)})$ , in which only the first factor is calibrated. It is exactly invariant under the rescaling group of the network (Proposition 1), recovers the fixed-architecture condition at the null (Theorem ??) uniformly over the unpenalized parameters, and is the largest admissible such rule (Proposition ??).
- The consequence that the calibrated threshold does not depend on the architecture (Corollary ??): one calibration serves every depth and width, and collapses to the linear-model threshold.
- Pivotal losses and closed-form null gradients for Gaussian regression, binary classification and the Cox model (Table 1), reducing the calibration to a single matrix product per Monte Carlo draw.
- Equivalent formulations of the penalized problem (Proposition 3): the penalty is a lasso on the first layer on the representatives where every sensitivity equals one, and the selected set is read off rescaling-invariant coefficients.
- An optimization scheme (Section 5) that fixes this gauge at every step and takes proximal adaptive steps on the resulting lasso, along a warm path in  $\lambda$ . Its fixed points are the first-order points of the problem, and at the null it performs exactly the calibrated zero-thresholding test (Proposition 8); we state what it does not guarantee.

## 2 Related Work

**Sparse-input networks.** Penalizing the input layer of a network to obtain feature selection is an established device. Feng and Simon (2019) fit sparse-input networks with a group penalty on the input weights, Louizos et al. (2018) relax an  $\ell_0$  count with stochastic gates, as do Yamada et al. (2020), and Lemhadri et al. (2021) tie the input layer to a residual linear path so that the selected support is nested along the path. In all of these the regularization level is a tuning parameter, chosen by cross-validation or by a target sparsity; the present paper leaves the architecture and the penalty alone and supplies the level.

**Selection with error control.** A second line of work attaches a frequentist guarantee directly to the selected set. Knockoffs (Barber and Candès, 2015) and their model-X extension (Candès et al., 2018) control the false discovery rate by comparing each original variable with an artificial counterpart designed to mimic the dependence structure of the covariates while carrying no additional information about the response. DeepPINK (Lu

et al., 2018) extends this principle to neural networks through a dedicated architecture that jointly processes original variables and their knockoffs. These approaches provide a stronger inferential guarantee than the one pursued here, but require the construction of valid knockoff variables, which in the model-X setting relies on knowledge or accurate estimation of the covariate distribution. Our approach instead operates directly on the original covariates and can be integrated into a standard MLP without generating artificial variables or modifying the architecture to accommodate them. The resulting guarantee is different in nature: our calibration controls the zero-thresholding behavior at the null model rather than the false discovery rate of the returned support.

**Tuning without cross-validation.** Choosing  $\lambda$  from the null rather than from held-out risk goes back to the universal threshold of Donoho and Johnstone (1994) and, for the lasso, to the theoretical choice of Bickel et al. (2009). The quantile universal threshold (Giacobino et al., 2017) replaces the asymptotic rate by a finite-sample quantile of the zero-thresholding statistic, and the pivotal information criterion (Sardy et al., 2026) makes that quantile computable by removing the nuisance parameters from the statistic. This paper is the extension of that programme to a nonlinear, non-identified model.

**Rescaling invariance.** That a positively homogeneous network can move mass between layers at no cost, so that a norm read off one layer means nothing on its own, is the phenomenon behind path-normalized measures of capacity (Neyshabur et al., 2015b). It affects optimization as well: gradient descent is not invariant under these rescalings, and Path-SGD (Neyshabur et al., 2015a) replaces it by an approximate steepest descent in the geometry of the path norm, which is invariant. We take a different route, dictated by what the calibration needs: rather than replacing the penalty by an invariant one, we keep the  $\ell_1$  penalty on  $W^{(1)}$  and rescale the *level*, by exactly the factor that appears in the null gradient. The resulting cost is invariant and the threshold retains its meaning. Rather than changing the metric, the optimization then fixes a gauge.

Optimizing over the orbits of a symmetry through representatives, with algorithms that must not depend on the representative, is the framework of optimization on quotient manifolds (Absil et al., 2008). Several methods interleave descent steps with free moves along the orbits of the rescalings. Equi-normalization (Stock et al., 2019) rescales every neuron, between two steps of stochastic gradient descent and without changing the function, towards the unique representative that minimizes the  $\ell_2$  norm of the weights, computed by iterations inspired by the Sinkhorn–Knopp algorithm. Symmetry teleportation (Zhao et al., 2022) moves along a level set of the loss, by a group action that leaves it invariant, to a point where the gradient is steeper, and gives a necessary condition for this to accelerate convergence. Our retraction is such a move at every step, with an explicit target: the representative is dictated by the penalty rather than by a norm, and it is computed neuron by neuron in closed form. Weight normalization (Salimans and Kingma, 2016) fixes the scale by construction, writing each weight vector as  $g\mathbf{v}/\|\mathbf{v}\|$ ; no such reparameterization is available here (Section 5.3). Without any gauge, gradient flow conserves, neuron by neuron, the difference between the squared norms of the incoming and of the outgoing weights (Du et al., 2018), which is what lets the scales drift.

The closest algorithm to ours is PathProx (Yang et al., 2022). For ReLU networks, weight decay is equivalent to a penalty  $\sum_k \|\mathbf{w}_k\|_2 \|\mathbf{v}_k\|_2$  on the input and output weights of

every neuron, and PathProx normalizes the input weights of each neuron, a gauge, and takes proximal steps on the output weights, a group soft-thresholding. Its gauge factor  $\|\mathbf{w}_k\|_2$  has a radial gradient, along the orbit, which the normalization absorbs. Ours,  $s_k$ , depends on the data and on the deeper layers, and its gradient must be kept (Section 5.2): the present method is a PathProx whose gauge factor depends on the data. Its first-layer step is an instance of the proximal adaptive methods of Yun et al. (2021), whose proximal operator is taken in the metric of the preconditioner, and of variable-metric forward-backward splitting (Combettes and Vũ, 2014).

### 3 Sparse Learning with Neural Networks

#### 3.1 Setting and Notation

Let  $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$  be independent observations, with  $\mathbf{x}_i \in \mathbb{R}^p$ , and let  $X \in \mathbb{R}^{n \times p}$  denote the design matrix. Throughout, without loss of generality, we normalize

$$\frac{1}{n} \sum_{i=1}^n x_{ij}^2 = 1 \quad (j = 1, \dots, p). \quad (1)$$

Consider a fully connected artificial neural network (ANN), also known as a multilayer perceptron (MLP), with  $L$  layers, defining a class of functions

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = S_L \circ S_{L-1} \circ \dots \circ S_1(\mathbf{x}).$$

Throughout this work, we restrict attention to MLPs with a scalar output, so that  $f_{\boldsymbol{\theta}} : \mathbb{R}^p \rightarrow \mathbb{R}$ . The parameter vector  $\boldsymbol{\theta}$  collects all model parameters. For observation  $i$ , we write  $\eta_i = f_{\boldsymbol{\theta}}(\mathbf{x}_i)$ , and  $\boldsymbol{\eta} = f_{\boldsymbol{\theta}}(X) \in \mathbb{R}^n$  for the vector of outputs over the  $n$  observations. Write  $p_0 = p$  for the input dimension and  $p_l$  for the number of neurons in layer  $l$ , so that  $W^{(l)} \in \mathbb{R}^{p_l \times p_{l-1}}$  and  $\mathbf{b}^{(l)} \in \mathbb{R}^{p_l}$  denote the weight matrix and bias of layer  $l$ . For  $l \in \{1, \dots, L-1\}$ , layer  $l$  applies an affine map followed by a componentwise nonlinearity,

$$S_l(\mathbf{u}) = \sigma_l(\mathbf{b}^{(l)} + W^{(l)}\mathbf{u}),$$

mapping the  $p_{l-1}$ -vector  $\mathbf{u}$  into a  $p_l$ -vector. The output layer is purely affine,

$$S_L(\mathbf{u}) = \mathbf{b}^{(L)} + W^{(L)}\mathbf{u}, \quad p_L = 1,$$

with no activation applied: any transformation of the output, such as a sigmoid for binary classification, is absorbed into the loss so that regression ( $y \in \mathbb{R}$ ), binary classification ( $y \in \{0, 1\}$ ) and survival data are handled by the same architecture.

With an MLP, the relevant features are the columns of the first-layer weight matrix  $W^{(1)}$  that are non-zero, as these are the only ones directly connected to the input  $X$ . For this identification to be meaningful, a small entry of  $W^{(1)}$  must not be reproducible by inflating weights in deeper layers: otherwise an input could influence the output while its first-layer column is arbitrarily small, severing the link between  $W^{(1)}$  and feature relevance and nullifying any penalty applied to  $W^{(1)}$ .

The quantity that governs this amplification is the sensitivity of the output to the first layer. Write  $\mathbf{h}_i^{(1)} = \mathbf{b}^{(1)} + W^{(1)}\mathbf{x}_i$  for the first-layer preactivation at observation  $i$  and define the *sensitivity vectors*

$$\mathbf{a}_i(\boldsymbol{\theta}) = \frac{\partial \eta_i}{\partial \mathbf{h}_i^{(1)}} = D_i^{(1)} W^{(2)\top} \dots D_i^{(L-1)} W^{(L)\top} \in \mathbb{R}^{p_1}, \quad i = 1, \dots, n. \quad (2)$$

where  $D_i^{(\ell)} = \text{diag}(\sigma'_\ell(\mathbf{h}_i^{(\ell)}))$ . They measure how strongly the network output responds to each first-layer unit, and they are the exact factor by which a gradient signal is amplified between the penalized weights and the output. For each first-layer neuron  $k$ , its *neuron sensitivity scale* is defined by

$$s_k(\boldsymbol{\theta}) = \|\mathbf{a}_k(\boldsymbol{\theta})\|_\infty = \max_{1 \leq i \leq n} |a_{ik}(\boldsymbol{\theta})|. \quad (3)$$

Section 3.2 shows that measuring the penalty on this sensitivity scale removes the arbitrary neuron-wise scaling of the network.

Throughout this work, we use the leaky ReLU activation

$$\sigma_\ell(t) = \max\{t, mt\}, \quad \ell = 1, \dots, L-1, \quad m \in (0, 1),$$

in all hidden layers. This choice is motivated by two properties that are particularly useful for the proposed regularization.

First, the derivative of each hidden-layer activation remains bounded away from zero wherever it is defined. More precisely,

$$|\sigma'_\ell(t)| \geq m > 0,$$

This prevents the activation functions themselves from completely blocking the propagation of information through the network and, in particular, the backpropagated signal used to construct the sensitivity vectors  $\mathbf{a}_i$ . This property is also relevant to the structural interpretation of variable selection through  $W^{(1)}$ . If an intermediate activation contains a flat region, an input-output path may be blocked by the activation even though the corresponding entries of  $W^{(1)}$  are nonzero. The network may then become functionally independent of some input variables without the associated first-layer weights being zero.

Second, the leaky ReLU is positively homogeneous:

$$\sigma_\ell(ct) = c \sigma_\ell(t), \quad c > 0.$$

Although this property holds for every hidden layer, it is specifically required at the first one for the scale-invariance of the adaptive penalty developed in Section 3.2.

### 3.2 The Scale Ambiguity and the Adaptive Regularization

A multilayer network does not uniquely identify its own parameters, and this ambiguity affects precisely the quantity that a penalty on  $W^{(1)}$  is meant to measure. By positive homogeneity of the first-layer activation, for any first-layer neuron  $k$  and any  $c > 0$ , the reparameterization

$$W_{k,\cdot}^{(1)} \mapsto c W_{k,\cdot}^{(1)}, \quad b_k^{(1)} \mapsto c b_k^{(1)}, \quad W_{\cdot,k}^{(2)} \mapsto \frac{1}{c} W_{\cdot,k}^{(2)} \quad (4)$$

leaves the fitted function  $f_{\boldsymbol{\theta}}$ , and therefore the loss, unchanged, while multiplying  $\|W_{k,\cdot}^{(1)}\|_1$  by  $c$ . Read in the direction  $c \rightarrow 0$ , this is the failure mode announced in Section 3.1: the network drives its input weights towards zero, compensates by inflating  $W^{(2)}$ , predicts exactly what it predicted before, and pays a penalty as small as it likes.

The sensitivity scale (3) transforms in the opposite direction. Indeed, under (4),

$$\mathbf{a}_k(\boldsymbol{\theta}) \mapsto \frac{1}{c} \mathbf{a}_k(\boldsymbol{\theta}), \quad s_k(\boldsymbol{\theta}) \mapsto \frac{1}{c} s_k(\boldsymbol{\theta}).$$

The sensitivity scale therefore provides exactly the compensating factor needed to remove this arbitrary parameter scaling.

**Proposition 1** *The adaptive first-layer penalty*

$$\mathcal{P}(\boldsymbol{\theta}) = \sum_{k=1}^{p_1} s_k(\boldsymbol{\theta}) \|W_{k,\cdot}^{(1)}\|_1 \quad (5)$$

*is invariant under any positive neuron-wise reparameterization of the form (4). Consequently, any penalized criterion does not depend on the arbitrary scale chosen between a first-layer neuron and the following layer.*

The role of the sensitivity scale extends beyond scale invariance. By matching the sensitivity factor appearing in the first-layer gradient, it removes neuron-specific downstream amplification from the regularization threshold, thereby enabling the calibration of a single regularization parameter  $\lambda$ , as developed in Section 4.

**The rescaling group.** Write  $\mathcal{G} = (0, \infty)^{p_1}$  and, for  $\mathbf{c} \in \mathcal{G}$ , let  $G_{\mathbf{c}}$  apply (4) to every first-layer neuron  $k$  with  $c = c_k$ . These rescalings compose,  $G_{\mathbf{c}} G_{\mathbf{c}'} = G_{\mathbf{c} \odot \mathbf{c}'}$ . They multiply  $\mathbf{h}_i^{(1)}$  entrywise by  $\mathbf{c}$  and leave every deeper pre-activation unchanged, so that they change no activation pattern, and

$$f_{G_{\mathbf{c}}\boldsymbol{\theta}} = f_{\boldsymbol{\theta}}, \quad s_k(G_{\mathbf{c}}\boldsymbol{\theta}) = \frac{s_k(\boldsymbol{\theta})}{c_k}, \quad k = 1, \dots, p_1; \quad (6)$$

rescaling neuron  $k$  leaves every  $s_j$ ,  $j \neq k$ , unchanged. Let  $\Theta_+ = \{\boldsymbol{\theta} : s_k(\boldsymbol{\theta}) > 0, k = 1, \dots, p_1\}$ .

**Lemma 2 (Canonical representative)** *For every  $\boldsymbol{\theta} \in \Theta_+$ , the orbit  $\{G_{\mathbf{c}}\boldsymbol{\theta} : \mathbf{c} \in \mathcal{G}\}$  meets the set  $\Sigma = \{\boldsymbol{\theta} \in \Theta_+ : s(\boldsymbol{\theta}) = \mathbf{1}\}$  at exactly one point,  $G_{s(\boldsymbol{\theta})}\boldsymbol{\theta}$ .*

**Proof** By (6),  $s(G_{\mathbf{c}}\boldsymbol{\theta}) = \mathbf{1}$  if and only if  $c_k = s_k(\boldsymbol{\theta})$  for every  $k$ . ■

### 3.3 The Learning Problem

Given data  $(X, \mathbf{y})$ , we consider the training problem

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^\gamma} J_\lambda(\boldsymbol{\theta}), \quad J_\lambda(\boldsymbol{\theta}) = \ell(\boldsymbol{\theta}) + \lambda \sum_{k=1}^{p_1} s_k(\boldsymbol{\theta}) \|W_{k,\cdot}^{(1)}\|_1, \quad \ell(\boldsymbol{\theta}) = \mathcal{L}_n(f_{\boldsymbol{\theta}}(X); \mathbf{y}), \quad (\text{P})$$

where  $\mathcal{L}_n(\boldsymbol{\eta}; \mathbf{y})$  is a data-dependent goodness-of-fit measure of the vector of outputs, and  $\lambda \geq 0$  is the regularization parameter controlling the strength of the adaptive first-layer penalty, set to the level  $\lambda_\alpha^{\text{DB}}$  developed in Section 4. The full expression  $J_\lambda$  is referred to as the cost function. Throughout this work, we consider three statistical learning settings: regression, binary classification, and survival analysis under the Cox proportional hazards model. In each case, the goodness-of-fit term  $\mathcal{L}_n$  is chosen with a specific normalization that will play a central role in the calibration of  $\lambda$ . We consider the same three losses as Sardy et al. (2026).

For regression, with  $\mathbf{y} \in \mathbb{R}^n$ , we use the square-root loss

$$\mathcal{L}_n^{\text{reg}}(\boldsymbol{\eta}; \mathbf{y}) = \frac{1}{\sqrt{n}} \|\mathbf{y} - \boldsymbol{\eta}\|_2.$$

For binary classification, with  $y_i \in \{0, 1\}$  and  $\mu_i = 1/(1 + e^{-\eta_i})$  denoting the logit produced by the network, we use the transformed logistic loss

$$\mathcal{L}_n^{\text{log}}(\boldsymbol{\eta}; \mathbf{y}) = \frac{2}{n} \sum_{i=1}^n \left[ y_i \sqrt{\frac{1 - \mu_i}{\mu_i}} + (1 - y_i) \sqrt{\frac{\mu_i}{1 - \mu_i}} \right].$$

For survival analysis, let  $(y_i, \delta_i)$  denote the observed time and event indicator for subject  $i$ . We use the square-root negative Cox partial log-likelihood

$$\mathcal{L}_n^{\text{Cox}}(\boldsymbol{\eta}; \mathbf{T}, \boldsymbol{\delta}) = \sqrt{\sum_{i=1}^n \delta_i \left[ \eta_i - \log \left( \sum_{j: y_j \geq y_i} e^{\eta_j} \right) \right]}.$$

These particular forms and normalizations are not merely conventional choices. They are selected so that the score of the loss at the null model is a pivotal quantity, free of any nuisance parameters. This property is what ultimately allows our choice of regularization level  $\lambda_\alpha^{\text{DB}}$  developed in Section 4.

The numerical solution of (P) is considered separately in Section 5. In particular, we discuss why the final regularization level should be approached along a regularization path rather than imposed directly at initialization, and describe the optimization scheme used to solve the successive penalized problems.

### 3.4 Equivalent Formulations

By Proposition 1,  $J_\lambda \circ G_{\mathbf{c}} = J_\lambda$  for every  $\mathbf{c} \in \mathcal{G}$ , and Lemma 2 selects one point on every orbit. Problem (P) can therefore be written in several equivalent ways.

- *Constrained form, or fixed gauge.* On the representatives  $\Sigma$ , the penalty is a lasso on the first layer:

$$\min_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}) + \lambda \|W^{(1)}\|_1 \quad \text{subject to} \quad s_k(\boldsymbol{\theta}) = 1, \quad k = 1, \dots, p_1. \quad (\text{C})$$

- *Quotient form.*  $J_\lambda$  is constant on the orbits,  $J_\lambda = \bar{J}_\lambda \circ \pi$  with  $\pi : \Theta_+ \rightarrow \Theta_+/\mathcal{G}$  the projection onto the orbits, and (P) on  $\Theta_+$  is the minimization of  $\bar{J}_\lambda$  over the

orbits. By Lemma 2,  $\Sigma$  meets every orbit once: it is a global section, a gauge, which identifies  $\Theta_+/\mathcal{G}$  with  $\Sigma$ . Since  $s$  is not continuous, this identification is one of sets, not of manifolds.

- *Invariant coefficients.* Let  $B_{k,\cdot} = s_k(\boldsymbol{\theta}) W_{k,\cdot}^{(1)}$ . Then  $B$  is invariant under  $\mathcal{G}$ ,  $J_\lambda = \ell + \lambda \|B\|_1$ , and  $B = W^{(1)}$  on  $\Sigma$ . The selected set  $\hat{S} = \{j : B_{\cdot,j} \neq \mathbf{0}\}$  is therefore a property of the orbit.  $B$  is not a free parameterization, since  $s$  depends on the data and on the activation patterns: it is a reading of the problem, not a change of variables.

Let  $\varphi$  denote the parameters other than  $W^{(1)}$ , namely every bias, including  $\mathbf{b}^{(1)}$ , and the weights  $W^{(2)}, \dots, W^{(L)}$ . Write  $\mathbf{g}_k = \nabla_{W_{k,\cdot}^{(1)}} \ell$ . Since  $J_\lambda$  is only piecewise smooth, first-order conditions are stated at regular points. We call  $\boldsymbol{\theta}$  *regular* if no hidden pre-activation vanishes at any observation and if, for every  $k$ , the observations attaining  $s_k(\boldsymbol{\theta})$  share their activation patterns. Near a regular point the patterns are constant,  $\ell$  is differentiable, and  $s_k$  is differentiable, independent of  $W^{(1)}$  and of the biases, and depends on  $W^{(2)}$  only through its column  $W_{\cdot,k}^{(2)}$ . A regular  $\boldsymbol{\theta}$  is a *first-order point* of (P) if

$$\mathbf{0} \in \mathbf{g}_k(\boldsymbol{\theta}) + \lambda s_k(\boldsymbol{\theta}) \partial \|W_{k,\cdot}^{(1)}\|_1 \quad (k = 1, \dots, p_1), \quad \nabla_\varphi J_\lambda(\boldsymbol{\theta}) = \mathbf{0}, \quad (7)$$

and a first-order point of (C) if  $\boldsymbol{\theta} \in \Sigma$ , the first condition of (7) holds, and  $\nabla_\varphi \ell(\boldsymbol{\theta}) + \sum_k \nu_k \nabla_\varphi s_k(\boldsymbol{\theta}) = \mathbf{0}$  for some multipliers  $\nu \in \mathbb{R}^{p_1}$ .

**Proposition 3 (Equivalent formulations)** *For every  $\lambda \geq 0$ :*

- (i) *Problems (P), restricted to  $\Theta_+$ , and (C) have the same infimum, and  $\boldsymbol{\theta} \mapsto G_{s(\boldsymbol{\theta})}\boldsymbol{\theta}$  maps the global minimizers of the first onto those of the second.*
- (ii) *A regular  $\boldsymbol{\theta} \in \Theta_+$  is a first-order point of (P) if and only if  $G_{s(\boldsymbol{\theta})}\boldsymbol{\theta}$  is a first-order point of (C), whose multipliers are then  $\nu_k = \lambda \|W_{k,\cdot}^{(1)}\|_1$ .*
- (iii)  *$B$  is invariant under  $\mathcal{G}$ ,  $J_\lambda = \ell + \lambda \|B\|_1$ , and  $B = W^{(1)}$  on  $\Sigma$ .*

The proof is given in Appendix A. At regular points of  $\Sigma$ , the gradients  $\nabla_\varphi s_k$  are linearly independent, so that the first-order conditions of (C) are necessary for its local minimizers. Problem (C) is the one the optimization of Section 5 works on, and the one where the calibrated level has the meaning of a lasso threshold (Section 4.2).

## 4 Choice of $\lambda$

### 4.1 Calibration under the null

The regularization parameter  $\lambda$  in (P) governs the sparsity of the learned model: if  $\lambda$  is too large, the penalty dominates the loss and drives  $W^{(1)} = \mathbf{0}$ , thereby discarding all input features; if  $\lambda$  is too small, irrelevant features may enter the model. A standard approach is to select  $\lambda$  by cross-validation. This is unattractive in the present setting for two reasons.

First, repeated fitting is computationally expensive for neural networks. Second, cross-validation targets predictive performance rather than support recovery and may therefore favour insufficient regularization when the objective is variable selection.

We instead build on the null-calibration principle of Sardy et al. (2026). The central idea is to calibrate a reference regularization level under the pure-noise model  $H_0 : W^{(1)} = \mathbf{0}$  so that, with prescribed probability  $1 - \alpha$ , the penalized criterion (P) admits  $W^{(1)} = \mathbf{0}$  as a local minimum.

In a multilayer model, the gradient with respect to  $W^{(1)}$  depends not only on the statistical association between  $X$  and  $\mathbf{y}$ , but also on how strongly the downstream network propagates perturbations of the first layer to the output. These two contributions play fundamentally different roles. The former is determined by the data and can be calibrated under  $H_0$ , whereas the latter depends on the parametrization of the downstream network and may evolve during training. However, the adaptive first-layer penalty (5) is constructed using precisely the same sensitivity factor that enters the first-layer gradient. At the null model, this factor therefore cancels from the local optimality condition, removing the neuron-specific effect of the downstream parametrization. The resulting zero-threshold condition depends only on the data-dependent score and can thus be calibrated through a single regularization parameter  $\lambda$ .

The following theorem turns the requirement that  $W^{(1)} = \mathbf{0}$  be a local minimizer into a condition on  $\lambda$ . Consider the null parameter  $\boldsymbol{\theta}^0 = (W^{(1)} = \mathbf{0}, \hat{\boldsymbol{\tau}})$ , where  $\hat{\boldsymbol{\tau}}$  minimizes the loss subject to  $W^{(1)} = \mathbf{0}$ . Under this constraint, the network output is constant across observations, so that  $\hat{\boldsymbol{\tau}}$  yields the optimal constant fit under  $\mathcal{L}_n$ . For each first-layer neuron  $k = 1, \dots, p_1$ , let

$$\mathbf{g}_{0,k} = \nabla_{W_{k,\cdot}^{(1)}} \mathcal{L}_n(f_{\boldsymbol{\theta}}(X); \mathbf{y}) \Big|_{\boldsymbol{\theta}=\boldsymbol{\theta}^0} \in \mathbb{R}^p \quad (8)$$

denote the corresponding first-layer gradient at the null model, and

$$\hat{\mathbf{r}}_0(\mathbf{y}) = \nabla_{\boldsymbol{\eta}} \mathcal{L}_n(\boldsymbol{\eta}; \mathbf{y}) \Big|_{\boldsymbol{\eta}=f_{\boldsymbol{\theta}^0}(X)} \in \mathbb{R}^n \quad (9)$$

the gradient of the loss with respect to the network output, evaluated at the optimal constant fit.

**Theorem 4 (Zero-thresholding function)** *Assume that  $s_k(\boldsymbol{\theta}^0) > 0$  for all  $k = 1, \dots, p_1$ . The null parameter  $\boldsymbol{\theta}^0 = (W^{(1)} = \mathbf{0}, \hat{\boldsymbol{\tau}})$  is a local minimizer of the cost function (P) if and only if*

$$\lambda \geq \max_{k=1,\dots,p_1} \frac{\|\mathbf{g}_{0,k}\|_{\infty}}{s_k(\boldsymbol{\theta}^0)} = \|X^T \hat{\mathbf{r}}_0(\mathbf{y})\|_{\infty} =: \lambda_0(X, \mathbf{y}).$$

**Proof** At  $W^{(1)} = \mathbf{0}$ , the first-layer preactivation is identical across observations,  $\mathbf{h}_i^{(1)} = \mathbf{b}^{(1)}$  for all  $i = 1, \dots, n$ . Since the downstream network therefore receives the same input for every observation, the first-layer sensitivity of neuron  $k$  is constant across the sample. Hence, for some scalar  $a_{0k}$ ,

$$\mathbf{a}_{\cdot k}(\boldsymbol{\theta}^0) = a_{0k} \mathbf{1}_n, \quad s_k(\boldsymbol{\theta}^0) = |a_{0k}|.$$

Let  $\hat{\mathbf{r}}_0(\mathbf{y}) = \nabla_{\boldsymbol{\eta}} \mathcal{L}_n(\boldsymbol{\eta}; \mathbf{y}) \Big|_{\boldsymbol{\theta}=\boldsymbol{\theta}^0}$  denote the loss gradient at the optimal constant fit. By the chain rule, the gradient with respect to the  $k$ th row of the first-layer weights is

$$\mathbf{g}_{0,k} = X^{\top} (\mathbf{a}_{\cdot k}(\boldsymbol{\theta}^0) \odot \hat{\mathbf{r}}_0(\mathbf{y})) = a_{0k} X^{\top} \hat{\mathbf{r}}_0(\mathbf{y}).$$

Therefore,

$$\frac{\|\mathbf{g}_{0,k}\|_\infty}{s_k(\boldsymbol{\theta}^0)} = \frac{|a_{0k}| \|X^\top \hat{\mathbf{r}}_0(\mathbf{y})\|_\infty}{|a_{0k}|} = \|X^\top \hat{\mathbf{r}}_0(\mathbf{y})\|_\infty.$$

This quantity is identical for every first-layer neuron. Finally, the subgradient condition associated with the adaptive penalty at  $W_{k,\cdot}^{(1)} = \mathbf{0}$  is  $\|\mathbf{g}_{0,k}\|_\infty \leq \lambda s_k(\boldsymbol{\theta}^0)$  for  $k = 1, \dots, p_1$ . Since  $s_k(\boldsymbol{\theta}^0) > 0$ , these conditions hold simultaneously if and only if  $\lambda \geq \max_{1 \leq k \leq p_1} \frac{\|\mathbf{g}_{0,k}\|_\infty}{s_k(\boldsymbol{\theta}^0)}$  yielding the provided statement.  $\blacksquare$

The zero-thresholding function  $\lambda_0(X, \mathbf{y})$  gives the critical regularization level at which  $W^{(1)} = \mathbf{0}$  becomes locally optimal for the observed data  $(X, \mathbf{y})$ . To turn this data-dependent threshold into a calibrated regularization parameter, we evaluate its distribution in the setting where the null solution is appropriate, namely under  $H_0$ . This null distribution provides the reference against which the regularization level is calibrated.

**Definition 5 (Detection boundary)** *Let  $\mathbf{Y}_0$  be a response vector generated under the pure-noise model. Define the random variable  $\Lambda = \lambda_0(X, \mathbf{Y}_0)$ , and let  $F_\Lambda$  denote its cumulative distribution function. Then, for a prescribed  $\alpha \in (0, 1)$ , the  $\alpha$ -level detection boundary is*

$$\lambda_\alpha^{\text{DB}} = F_\Lambda^{-1}(1 - \alpha). \quad (10)$$

The resulting  $\lambda_\alpha^{\text{DB}}$  provides a fixed reference threshold calibrated under the null model. By construction,  $\mathbb{P}_{H_0} \{ \lambda_0(X, \mathbf{Y}_0) \leq \lambda_\alpha^{\text{DB}} \mid X \} = 1 - \alpha$ . Combined with Theorem 4, this implies that  $\lambda_\alpha^{\text{DB}}$  is the regularization level at which the null parameter satisfies the zero-threshold condition with probability  $1 - \alpha$  under  $H_0$ . This threshold is determined prior to network optimization and remains unchanged throughout training. Importantly, the downstream network parametrization does not enter its calibration: its contribution to the first-layer gradient has already been removed by the sensitivity scaling in Theorem 4. The calibration of  $\lambda$  therefore depends only on the distribution of the zero-thresholding function under  $H_0$ .

In general, such a null distribution may depend on unknown nuisance parameters, preventing the detection boundary from being calibrated without their estimation. The losses introduced in Section 3.3 are chosen precisely to avoid this difficulty. For each of the three learning tasks considered here, the corresponding zero-thresholding statistic  $\Lambda = \lambda_0(X, \mathbf{Y}_0)$  is pivotal under  $H_0$ , meaning that its null distribution is free of unknown nuisance parameters of  $\mathbf{Y}_0$ . Consequently, the detection boundary can be approximated by Monte Carlo simulation under any convenient choice of these nuisance parameters: repeated null samples  $\mathbf{Y}_0^{(1)}, \dots, \mathbf{Y}_0^{(M)}$  are generated, the corresponding zero-thresholding statistics are computed, and  $\lambda_\alpha^{\text{DB}}$  is estimated by their empirical  $(1 - \alpha)$ -quantile.

Table 1 summarizes the resulting zero-thresholding statistic for regression, binary classification, and survival analysis. Further details on the pivotal construction are provided in Sardy et al. (2026), with the survival analysis setting developed specifically in van Cutsem and Sardy (2026).

| Learning task         | Zero-thresholding statistic $\lambda_0(X, \mathbf{y})$                                                                                                          |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Regression            | $\frac{\ X^\top(\mathbf{y} - \bar{\mathbf{y}}\mathbf{1})\ _\infty / \sqrt{n}}{\ \mathbf{y} - \bar{\mathbf{y}}\mathbf{1}\ _2}$                                   |
| Binary classification | $\frac{\ X^\top(\mathbf{y} - \bar{\mathbf{y}}\mathbf{1})\ _\infty / n}{\sqrt{\bar{\mathbf{y}}(1 - \bar{\mathbf{y}})}}$                                          |
| Survival analysis     | $\frac{\ n^{-1} \sum_{i=1}^n \delta_i(\mathbf{x}_i - \bar{\mathbf{x}}_{\mathcal{R}_i})\ _\infty}{2\{n^{-1} \sum_{i=1}^n \delta_i \log  \mathcal{R}_i \}^{1/2}}$ |

Table 1: Pivotal zero-thresholding statistics for the three learning tasks considered in this work. For the survival analysis task,  $\mathcal{R}_i = \{j : y_j \geq y_i\}$  and  $\bar{\mathbf{x}}_{\mathcal{R}_i} = |\mathcal{R}_i|^{-1} \sum_{j \in \mathcal{R}_i} \mathbf{x}_j$

## 4.2 Interpretation away from the null model

The calibration above is derived at the null model  $W^{(1)} = \mathbf{0}$ , where the sensitivity of each first-layer neuron is constant across observations and cancels exactly from the zero-threshold condition. We now consider the role of the same regularization parameter away from the null model.

For a general parameter  $\boldsymbol{\theta}$  with  $W^{(1)} \neq \mathbf{0}$ , define the normalized sensitivity vector

$$\tilde{\mathbf{a}}_k(\boldsymbol{\theta}) = \frac{\mathbf{a}_k(\boldsymbol{\theta})}{s_k(\boldsymbol{\theta})}, \quad \|\tilde{\mathbf{a}}_k(\boldsymbol{\theta})\|_\infty = 1,$$

for every neuron such that  $s_k(\boldsymbol{\theta}) > 0$ . This decomposition separates two distinct components of the downstream sensitivity:  $s_k(\boldsymbol{\theta})$  captures its overall magnitude, whereas  $\tilde{\mathbf{a}}_k(\boldsymbol{\theta})$  preserves its variation across observations. In particular, the former measures how strongly neuron  $k$  can affect the network output, while the latter describes how this influence is distributed across the sample.

Let  $\mathbf{r}(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\eta}} \mathcal{L}_n(\boldsymbol{\eta}; \mathbf{y})|_{\boldsymbol{\eta}=f_{\boldsymbol{\theta}}(X)}$  denote the loss gradient with respect to the network output. By the chain rule, the gradient associated with the  $k$ th first-layer neuron satisfies

$$\mathbf{g}_k(\boldsymbol{\theta}) = X^\top (\mathbf{a}_k(\boldsymbol{\theta}) \odot \mathbf{r}(\boldsymbol{\theta})) = s_k(\boldsymbol{\theta}) X^\top (\tilde{\mathbf{a}}_k(\boldsymbol{\theta}) \odot \mathbf{r}(\boldsymbol{\theta})).$$

Consequently, for each first-layer connection  $(k, j)$ , the corresponding threshold condition can be written as

$$|g_{kj}(\boldsymbol{\theta})| \leq \lambda s_k(\boldsymbol{\theta}) \iff \left| X_j^\top (\tilde{\mathbf{a}}_k(\boldsymbol{\theta}) \odot \mathbf{r}(\boldsymbol{\theta})) \right| \leq \lambda.$$

Hence, the adaptive penalty removes the overall magnitude  $s_k(\boldsymbol{\theta})$  of the downstream sensitivity from the thresholding condition, while retaining its observation-specific structure through  $\tilde{\mathbf{a}}_k(\boldsymbol{\theta})$ . The regularization parameter  $\lambda$  therefore acts on a normalized first-layer score and can be shared across all first-layer neurons.

The normalized score  $X_j^\top (\tilde{\mathbf{a}}_k \odot \mathbf{r}) = g_{kj}/s_k$  is invariant under  $\mathcal{G}$ : it is the gradient of  $\ell$  with respect to the invariant coefficient  $B_{kj}$  of Section 3.4, at fixed  $s$ . In the constrained form (C), where  $s_k = 1$  and  $\tilde{\mathbf{a}}_k = \mathbf{a}_k$ , the condition reads  $|g_{kj}(\boldsymbol{\theta})| \leq \lambda$ : the zero-thresholding condition of a lasso at level  $\lambda$ , which the calibration of Section 4 sets at the null.

## 5 Optimization

### 5.1 A Reversed Warm Path

While the ultimate goal is to solve (P), the calibrated level  $\lambda_\alpha^{\text{DB}}$  should not in general be imposed directly at initialization. The reason is that  $\lambda_\alpha^{\text{DB}}$  is calibrated from the first-order behavior of the network at the null model. A relevant variable may nevertheless have little or no marginal association with the response when its effect is purely nonlinear. Consequently, its relevance may not be visible from the first-order score at the null model. Starting the optimization directly at  $\lambda_\alpha^{\text{DB}}$  may then drive the first-layer weights to zero before the network has learned the nonlinear representation through which that variable becomes informative.

We therefore take the conservative approach of solving for a sequence of  $\lambda$ 's in the spirit of simulated annealing. Namely, we propose solving (P) using an increasing grid

$$0 < \lambda^{(1)} < \lambda^{(2)} < \dots < \lambda^{(M)} = \lambda_\alpha^{\text{DB}}.$$

Taking as initial parameter values the solution corresponding to the previous  $\lambda^{(i)}$  leads to a sequence of sparser approximating solutions until solving for  $\lambda_\alpha^{\text{DB}}$  at the last step. Along this path, the normalized sensitivities  $\tilde{\mathbf{a}}_k(\boldsymbol{\theta})$  evolve with the learned representation. Hence, once nonlinear structure has been learned, the adaptive first-layer score  $X_j^\top (\tilde{\mathbf{a}}_k(\boldsymbol{\theta}) \odot \mathbf{r}(\boldsymbol{\theta}))$  can reflect information that is absent from the marginal score at the null model. Increasing  $\lambda$  then progressively removes first-layer connections that are no longer supported by these learned, observation-specific scores.

### 5.2 Structure of the Problem

The penalty of (P) acts on  $W^{(1)}$  through its  $\ell_1$  norm, and on  $\varphi$  through the scales  $s_k$  only. Near a regular point,  $s_k$  is linear in the column  $W_{\cdot k}^{(2)}$  and, the activation patterns being frozen, positively homogeneous of degree one in every  $W^{(l)}$ ,  $l \geq 2$ .  $J_\lambda$  is not continuous, however: when an activation flips at the observation attaining  $s_k$ , the scale  $s_k$  jumps, by up to a factor  $1/m$ . Differentiating  $J_\lambda \circ G_{\mathbf{c}} = J_\lambda$  at  $\mathbf{c} = \mathbf{1}$  gives the following identities.

**Lemma 6 (Euler identities)** *At every regular  $\boldsymbol{\theta}$  and for every first-layer neuron  $k$ ,*

$$\langle W_{k,\cdot}^{(1)}, \mathbf{g}_k \rangle + b_k^{(1)} \partial_{b_k^{(1)}} \ell - \langle W_{\cdot k}^{(2)}, \nabla_{W_{\cdot k}^{(2)}} \ell \rangle = 0, \quad (11)$$

$$\langle W_{\cdot k}^{(2)}, \nabla_{W_{\cdot k}^{(2)}} s_k \rangle = \langle W^{(l)}, \nabla_{W^{(l)}} s_k \rangle = s_k, \quad l = 2, \dots, L. \quad (12)$$

Since  $J_\lambda$  depends on  $\varphi$  through the scales, the chain rule gives

$$\nabla_\varphi J_\lambda = \nabla_\varphi \ell + \sum_{k=1}^{p_1} \mu_k \nabla_\varphi s_k, \quad \mu_k = \lambda \|W_{k,\cdot}^{(1)}\|_1, \quad (13)$$

whereas  $W^{(1)}$  receives  $\nabla_{W^{(1)}} \ell$  alone, its penalty being handled by a proximal step (Section 5.4). The second term of (13) cannot be dropped.

**Proposition 7 (No sparse fixed point without the multiplier)** *Let  $\lambda > 0$ , and let  $\theta$  be regular with  $s_k(\theta) > 0$  for every  $k$ . If  $\nabla_\varphi \ell(\theta) = \mathbf{0}$  and the first condition of (7) holds, then  $W^{(1)} = \mathbf{0}$ .*

A scheme that moves  $\varphi$  along  $\nabla_\varphi \ell$  alone therefore has no fixed point that selects a variable. For a single neuron,  $\eta = v\sigma(wx)$  with  $x = 1$ , the loss  $\frac{1}{2}(\eta - 2)^2$  and  $\lambda = 1$ , problem (P) reads  $\min_u \frac{1}{2}(u - 2)^2 + |u|$  in  $u = vw > 0$ , whose minimizer is  $u^* = 1$ ; gradient descent with the retraction of Section 5.3 but without the second term of (13) settles instead at the root of  $(u - 2)(1 + u^2) = -1$ ,  $u \approx 1.75$ .

The multiplier term has a direct reading in (C). The Lagrangian  $\ell + \lambda \|W^{(1)}\|_1 + \sum_k \nu_k(s_k - 1)$  is stationary in  $\varphi$  when  $\nabla_\varphi \ell + \sum_k \nu_k \nabla_\varphi s_k = \mathbf{0}$ , and the identities of Lemma 6 force  $\nu_k = \mu_k$  (Proposition 3). The term  $\mu_k \nabla_\varphi s_k$  is thus the normal of the constraint  $s_k = 1$ , weighted by its Lagrange multiplier, which is known in closed form: no linear system has to be solved.

### 5.3 Gauge-Fixed Proximal Gradient

Each iteration first moves the iterate along its orbit to its canonical representative, then takes a proximal adaptive step (Section 5.4):

$$\bar{\theta}^t = G_{s(\theta^t)} \theta^t, \quad \theta^{t+1} = \text{prox-Adam}(\bar{\theta}^t, \nabla J_\lambda(\bar{\theta}^t)). \quad (14)$$

By Lemma 2,  $s(\bar{\theta}^t) = \mathbf{1}$ : the retraction fixes the gauge, and every step is taken on (C). The retraction leaves  $J_\lambda$  unchanged, and three reasons make it necessary.

**Effective step.** A proximal step of size  $\delta$  on  $W^{(1)}$ ,  $W_{kj}^{(1)} \leftarrow \mathcal{S}_{\delta \lambda s_k}(W_{kj}^{(1)} - \delta g_{kj})$  with  $\mathcal{S}_\tau(x) = \text{sign}(x) \max(|x| - \tau, 0)$ , reads on the invariant coefficients

$$B_{kj} \leftarrow \mathcal{S}_{\delta s_k^2 \lambda}(B_{kj} - \delta s_k^2 \hat{g}_{kj}), \quad \hat{g}_{kj} = X_j^\top (\tilde{\mathbf{a}}_k \odot \mathbf{r}),$$

since  $s \mathcal{S}_\tau(x) = \mathcal{S}_{s\tau}(sx)$  for  $s > 0$  and  $g_{kj} = s_k \hat{g}_{kj}$ . The threshold on the normalized score stays  $\lambda$ , but the effective step is  $\delta s_k^2$ : a neuron whose scale becomes small freezes before its row reaches zero, and the neuron is left hanging, with variables falsely selected. On  $\Sigma$ , the step is the same for every neuron, and  $B = W^{(1)}$ .

**Drift of the scales.** Gradient flow conserves, neuron by neuron, the difference between the squared norms of the incoming and of the outgoing weights (Du et al., 2018, Theorem 2.1). The argument only uses the invariance of the objective under the rescalings, and applies to  $J_\lambda$  wherever it is differentiable. As the lasso shrinks  $W_{k,\cdot}^{(1)}$ , the column  $W_{\cdot,k}^{(2)}$  shrinks with it, and so does  $s_k$ , which is linear in that column. The retraction breaks this conservation law on purpose.

**Equivariance.** Euclidean gradients are not covariant under  $\mathcal{G}$ :  $\mathbf{g}_k(G_c \theta) = \mathbf{g}_k(\theta)/c_k$ . Without the retraction, the trajectory would depend on the arbitrary scale of the starting point. With it, and with the optimizer state transported as below, the iterates depend on the orbit only (Proposition 8), as an algorithm on a quotient should (Absil et al., 2008).

**Transport of the optimizer state.** Differentiating  $J_\lambda \circ G_c = J_\lambda$  shows that gradients are covariant: the gradient of a weight multiplied by  $c$  is divided by  $c$ , and that of a weight divided by  $c$  is multiplied by  $c$ . The retraction, with  $c_k = s_k(\boldsymbol{\theta}^t)$ , applies the same rule to the moments of Adam: for  $W_{k,\cdot}^{(1)}$  and  $b_k^{(1)}$ , the gradient and the first moment are divided by  $c_k$  and the second moment by  $c_k^2$ ; for  $W_{k,\cdot}^{(2)}$ , they are multiplied by  $c_k$  and  $c_k^2$ . The transport is exact,  $J_\lambda$  being invariant, and necessary: since  $s_k$  jumps, moments left at the old scale would meet gradients at the new one in an oversized step. A neuron with  $s_k = 0$  is left as it is.

Like equi-normalization (Stock et al., 2019) and symmetry teleportation (Zhao et al., 2022), the iteration interleaves descent steps with free moves along the orbits; here the target is dictated by the penalty, and computed neuron by neuron in closed form. Weight normalization (Salimans and Kingma, 2016) would fix the gauge by construction. Here, by Lemma 2, the only rescaling of a parameter  $\mathbf{u}$  onto  $\Sigma$  is  $G_{s(\mathbf{u})}$ , and since  $J_\lambda(G_{s(\mathbf{u})}\mathbf{u}) = J_\lambda(\mathbf{u})$ , the reparameterization  $\boldsymbol{\theta} = G_{s(\mathbf{u})}\mathbf{u}$  gives (P) back in  $\mathbf{u}$ : the gauge has to be imposed by retraction. PathProx (Yang et al., 2022) also fixes a gauge, but the gradient of its gauge factor  $\|\mathbf{w}_k\|_2$  is radial, along the orbit, and the normalization absorbs it. Here  $\nabla_{W_{k,\cdot}^{(2)}} s_k$  is not collinear to  $W_{k,\cdot}^{(2)}$ , and  $s_k$  depends on the deeper layers shared by all neurons: the multiplier term of (13) changes the fitted function, and must be kept.

#### 5.4 Adaptive Proximal Step

The common step size of the proximal gradient step is replaced by the diagonal preconditioning of Adam (Kingma and Ba, 2017), within the proximal framework of Yun et al. (2021). On  $W^{(1)}$ , the step has no first moment,  $\beta_1 = 0$ :

$$W_{kj}^{(1)} \leftarrow \mathcal{S}_{D_{kj}\lambda s_k}(W_{kj}^{(1)} - D_{kj} g_{kj}), \quad D_{kj} = \frac{\delta}{\sqrt{\hat{v}_{kj}} + \varepsilon} > 0, \quad (15)$$

where  $\hat{v}_{kj}$  is the bias-corrected exponential moving average of  $g_{kj}^2$ , and  $s_k = 1$  after the retraction. This is the proximal operator of  $\lambda s_k |\cdot|$  in the diagonal metric  $D^{-1}$ , a variable-metric forward-backward step (Combettes and Vũ, 2014). Three properties follow.

1. *Thresholding test.* A zero coordinate stays at zero if and only if  $|g_{kj}| \leq \lambda s_k$ :  $D_{kj}$  cancels.
2. *Fixed points.*  $W^{(1)}$  is left unchanged if and only if the first condition of (7) holds:  $g_{kj} = -\lambda s_k \text{sign } W_{kj}^{(1)}$  where  $W_{kj}^{(1)} \neq 0$ , and  $|g_{kj}| \leq \lambda s_k$  elsewhere.  $D$  only sets the speed.
3. *No momentum.* With a first moment, the test would bear on an average of past gradients instead of  $\mathbf{g}(\boldsymbol{\theta}^t)$ , and inertia would carry across zero a coordinate the proximal operator has just set to zero.

The other parameters take standard Adam steps,  $\beta_1 = 0.9$ , along  $\nabla_\varphi J_\lambda$  in (13). A fixed point of  $\varphi$  and of its first moment  $m$  satisfies  $m = \nabla_\varphi J_\lambda$  and  $m = \mathbf{0}$ , hence  $\nabla_\varphi J_\lambda = \mathbf{0}$ : momentum is harmless there, no thresholding acting on  $\varphi$ . A phase stops once  $|J_\lambda(\boldsymbol{\theta}^{t-50}) - J_\lambda(\boldsymbol{\theta}^t)| \leq \text{tol} \cdot J_\lambda(\boldsymbol{\theta}^t)$ :  $J_\lambda$  is the objective actually decreased, and the retraction leaves it unchanged.

## 5.5 Properties

**Proposition 8 (Properties of the iteration)** *Consider the iteration (14), acting on the parameters and on the moments of Adam.*

- (a) Exactness. *The retraction changes neither  $f_{\theta}$ , nor the activation patterns, nor  $J_{\lambda}$ .*
- (b) Equivariance. *Started from  $G_{\mathbf{c}}\theta$ , with its moments transported as in Section 5.3, the iteration produces the same iterates as started from  $\theta$ , from the first retraction on: the iterates depend on the orbit of  $\theta$  only.*
- (c) Fixed points. *A regular  $\theta \in \Sigma$  with a zero first moment on  $\varphi$  is a fixed point if and only if it is a first-order point of (P), or equivalently of (C).*
- (d) Calibration preserved. *Let the null parameter  $\theta^0 = (W^{(1)} = \mathbf{0}, \hat{\tau})$  be regular and on  $\Sigma$ , with a zero first moment on  $\varphi$ . Then  $\theta^0$  is a fixed point if and only if  $\lambda \geq \lambda_0(X, \mathbf{y})$ .*

Property (d) is the condition of Theorem 4: along the fit, the iteration performs exactly the zero-thresholding test that  $\lambda_{\alpha}^{\text{DB}}$  is calibrated for, and the calibrated level keeps its meaning. The proofs of this section are given in Appendix A.

**Limitations.**  $J_{\lambda}$  is not continuous. The convergence guarantees of ProxGen (Yun et al., 2021), which assume a loss with a Lipschitz gradient, and those of Davis et al. (2020), for locally Lipschitz functions, do not apply directly, and Adam itself may fail to converge (Reddi et al., 2018). What is guaranteed is the nature of the fixed points, (c) and (d); we claim no convergence theorem. Moreover, (c) concerns regular points: where a hidden pre-activation vanishes at some observation, or where observations with different activation patterns tie for  $s_k$ ,  $J_{\lambda}$  is not differentiable, and stationarity can only be understood in a generalized sense.

## 5.6 Computation

**Sensitivities.** Each  $\eta_i$  depends on  $\mathbf{x}_i$  only, whatever the loss, the Cox partial likelihood included. Hence  $G_i = \partial \ell / \partial \mathbf{h}_i^{(1)} = r_i \mathbf{a}_i$  with  $r_i = \partial \ell / \partial \eta_i$ , and the backward pass of the loss yields  $a_{ik} = G_{ik} / r_i$  wherever  $r_i \neq 0$ . The rows where  $r_i = 0$ , saturated probabilities for instance, must be recomputed from the activation patterns by (2); doing so for every row costs one forward pass of the layers above the first, with neither division nor branch. The scales  $s_k$  and the observations  $i_k^* = \arg \max_i |a_{ik}|$  attaining them follow.

**Multiplier term in one pass.** With the activation patterns frozen and the biases removed, the network at observation  $i$  is the linear map  $\mathbf{u} \mapsto \mathbf{a}_i^{\top} \mathbf{u}$  of the first-layer pre-activation. Feeding  $\mathbf{u}^{(k)} = \mu_k \text{sign}(a_{i_k^*}) \mathbf{e}_k$  on a virtual row copied from observation  $i_k^*$ , for  $k = 1, \dots, p_1$ , gives

$$\sum_{k=1}^{p_1} \mathbf{a}_{i_k^*}^{\top} \mathbf{u}^{(k)} = \sum_{k=1}^{p_1} \mu_k s_k,$$

whose gradient with respect to  $W^{(2)}, \dots, W^{(L)}$  is the multiplier term of (13). It costs one forward and one backward pass of the deeper layers on  $p_1$  virtual rows.

---

**Algorithm 1** Gauge-fixed proximal fit along the warm path

---

**Require:**  $X$  (centred, unit second moment),  $\mathbf{y}$ , loss  $\mathcal{L}_n$ , level  $\alpha$ , number of phases  $M$ , ratio  $\rho$ , step size  $\delta$ , tolerances  $\text{tol}_m$

- 1:  $\lambda_\alpha^{\text{DB}} \leftarrow (1 - \alpha)$ -quantile of  $\|X^\top \hat{\mathbf{r}}_0(\mathbf{Y}_0)\|_\infty$  over Monte Carlo draws  $\mathbf{Y}_0$  under  $H_0$
- 2:  $\lambda^{(1)} < \dots < \lambda^{(M)} = \lambda_\alpha^{\text{DB}}$ , geometric from  $\rho \lambda_\alpha^{\text{DB}}$
- 3: **for**  $m = 1, \dots, M$ , warm-started **do**
- 4:    $\mathcal{A} \leftarrow$  non-zero columns of  $W^{(1)}$ , and the zero ones that fail the zero-thresholding test at  $\lambda^{(m)}$
- 5:   **repeat**
- 6:     forward and backward pass of  $\mathcal{L}_n$  on the columns of  $\mathcal{A}$ :  $\nabla \ell(\boldsymbol{\theta})$ ,  $G = \partial \ell / \partial \mathbf{h}^{(1)}$ ,  $\mathbf{r} = \partial \ell / \partial \boldsymbol{\eta}$
- 7:      $a_{ik} \leftarrow G_{ik}/r_i$ , recomputed where  $r_i = 0$ ;  $s_k \leftarrow \max_i |a_{ik}|$ ;  $i_k^* \leftarrow \arg \max_i |a_{ik}|$
- 8:      $\nabla_\varphi J_\lambda \leftarrow \nabla_\varphi \ell + \sum_k \mu_k \nabla_\varphi s_k$ ,  $\mu_k = \lambda^{(m)} \|W_{k,\cdot}^{(1)}\|_1$  {linearized pass}
- 9:      $\boldsymbol{\theta} \leftarrow G_s \boldsymbol{\theta}$ ; transport the gradients and the moments of Adam {now  $s = \mathbf{1}$ }
- 10:     $W^{(1)} \leftarrow \mathcal{S}_{D\lambda^{(m)}}(W^{(1)} - D \nabla_{W^{(1)}} \ell)$ ,  $D = \delta/(\sqrt{\hat{v}} + \varepsilon)$ ,  $\beta_1 = 0$
- 11:     $\varphi \leftarrow \text{Adam}(\varphi, \nabla_\varphi J_\lambda)$
- 12:    **until**  $|J_\lambda(\boldsymbol{\theta}^{t-50}) - J_\lambda(\boldsymbol{\theta}^t)| \leq \text{tol}_m J_\lambda(\boldsymbol{\theta}^t)$
- 13:    **if**  $m = M$  and some dropped columns fail the test at  $\lambda_\alpha^{\text{DB}}$  **then**
- 14:     add them to  $\mathcal{A}$  and repeat the phase
- 15:    **end if**
- 16: **end for**
- 17: **return**  $\hat{\boldsymbol{\theta}}$  and  $\hat{S} = \{j : \hat{W}_{\cdot j}^{(1)} \neq \mathbf{0}\}$

---

**Cost.** One iteration costs about one epoch of a standard multilayer perceptron. The first layer, with  $O(np p_1)$  operations, dominates when  $p$  is large.

**Active set.** A column of  $W^{(1)}$  that is zero at the end of a phase is dropped from the next one, which computes with the other columns only. A dropped column  $j$  would stay at zero under a proximal step on the whole of  $W^{(1)}$  if and only if  $\max_k |X_j^\top G_{\cdot k}|/s_k \leq \lambda$ , the zero-thresholding test, at the cost of one product with the dropped columns. The test is run when every phase starts, at its level, and when the last phase ends, at  $\lambda_\alpha^{\text{DB}}$ : the columns that fail are taken back and the phase is run again. The fixed points of the reduced problem that pass the test are exactly those of the full problem; in particular, when every column has been dropped, the final test at  $\lambda_\alpha^{\text{DB}}$  is the test of Theorem 4. This is the active-set strategy of glmnet (Friedman et al., 2010), whose optimality checks also complete the strong rules of Tibshirani et al. (2012).

Algorithm 1 summarizes the procedure.

## 5.7 Optional Refitting on the Selected Model

After completion of the regularization path, the selected support is determined from the nonzero columns of  $\hat{W}^{(1)}$ . If desired, the network can subsequently be reduced to this selected set of input features, together with the removal of any hidden neurons that have become inactive. The dimensions of  $\hat{W}^{(2)}$  are adjusted accordingly. An optional final

training phase can then be performed on this reduced architecture without regularization so as to reduce any estimation bias introduced by the penalized fit.

## 6 Experiments

### Appendix A. Proofs

Throughout,  $\theta$  is regular. Near  $\theta$ , the activation patterns are constant, so that  $a_{ik}$  is a multilinear function of  $(W_{\cdot k}^{(2)}, W^{(3)}, \dots, W^{(L)})$ , linear in each argument, and does not depend on  $W^{(1)}$  nor on the biases. The observations attaining  $s_k$  share their patterns, hence the same function  $a_{ik}$ , so that near  $\theta$ ,  $s_k = |a_{i_k^* k}|$ , which is differentiable when  $s_k > 0$ .

**Proof of Lemma 6.** For every  $\mathbf{c} \in \mathcal{G}$ ,  $\ell(G_{\mathbf{c}}\theta) = \ell(\theta)$  by (6). The map  $\mathbf{c} \mapsto G_{\mathbf{c}}\theta$  is smooth, and its derivative in  $c_k$  at  $\mathbf{c} = \mathbf{1}$  is  $W_{k,\cdot}^{(1)}$  in the coordinates of  $W_{k,\cdot}^{(1)}$ ,  $b_k^{(1)}$  in that of  $b_k^{(1)}$ ,  $-W_{\cdot k}^{(2)}$  in those of  $W_{\cdot k}^{(2)}$ , and zero elsewhere. Differentiating at  $\mathbf{c} = \mathbf{1}$ , where  $\ell$  is differentiable, gives (11). Near  $\theta$ ,  $s_k = |a_{i_k^* k}|$  is positively homogeneous of degree one in  $W_{\cdot k}^{(2)}$  and in every  $W^{(l)}$ ,  $l \geq 3$ , and it depends on  $W^{(2)}$  through  $W_{\cdot k}^{(2)}$  only: (12) is Euler's identity for homogeneous functions.

**Proof of Proposition 7.** By (11) and  $\nabla_{\varphi}\ell(\theta) = \mathbf{0}$ ,  $\langle W_{k,\cdot}^{(1)}, \mathbf{g}_k \rangle = 0$ . The first condition of (7) gives  $g_{kj} = -\lambda s_k \text{sign } W_{kj}^{(1)}$  wherever  $W_{kj}^{(1)} \neq 0$ , hence  $\langle W_{k,\cdot}^{(1)}, \mathbf{g}_k \rangle = -\lambda s_k \|W_{k,\cdot}^{(1)}\|_1$ . Since  $\lambda s_k > 0$ ,  $W_{k,\cdot}^{(1)} = \mathbf{0}$  for every  $k$ .

**Proof of Proposition 3.** (i) For  $\theta \in \Theta_+$ , let  $\bar{\theta} = G_{s(\theta)}\theta \in \Sigma$ . By invariance, and since  $s(\bar{\theta}) = \mathbf{1}$ ,  $J_{\lambda}(\theta) = J_{\lambda}(\bar{\theta}) = \ell(\bar{\theta}) + \lambda \|\bar{W}^{(1)}\|_1$ , the objective of (C) at  $\bar{\theta}$ . Every feasible point of (C) lies in  $\Sigma \subset \Theta_+$  and is its own representative. Both problems therefore have the same infimum, and  $\theta$  minimizes (P) over  $\Theta_+$  if and only if  $\bar{\theta}$  minimizes (C).

(ii) The conditions (7) hold at  $\theta$  if and only if they hold at  $G_{\mathbf{c}}\theta$ , for every  $\mathbf{c} \in \mathcal{G}$ . Indeed,  $G_{\mathbf{c}}\theta$  is regular,  $\mathbf{g}_k(G_{\mathbf{c}}\theta) = \mathbf{g}_k(\theta)/c_k$  and  $s_k(G_{\mathbf{c}}\theta) = s_k(\theta)/c_k$ , while  $\partial\|\cdot\|_1$  is unchanged by a positive rescaling of its argument; and differentiating  $J_{\lambda}(G_{\mathbf{c}}\theta) = J_{\lambda}(\theta)$  shows that  $\nabla_{\varphi}J_{\lambda}(G_{\mathbf{c}}\theta)$  is obtained from  $\nabla_{\varphi}J_{\lambda}(\theta)$  by multiplying its entries by positive factors. It is therefore enough to consider  $\theta \in \Sigma$ , where the first condition of (7) is common to both problems. If  $\nabla_{\varphi}J_{\lambda}(\theta) = \mathbf{0}$ , then, by (13), the stationarity condition of (C) holds with  $\nu_k = \mu_k$ . Conversely, let  $\nabla_{\varphi}\ell + \sum_j \nu_j \nabla_{\varphi}s_j = \mathbf{0}$ , and take its inner product with the vector  $\mathbf{t}_k$  equal to  $b_k^{(1)}$  in the coordinate of  $b_k^{(1)}$ , to  $-W_{\cdot k}^{(2)}$  in those of  $W_{\cdot k}^{(2)}$ , and to zero elsewhere. By (11),  $\langle \mathbf{t}_k, \nabla_{\varphi}\ell \rangle = -\langle W_{k,\cdot}^{(1)}, \mathbf{g}_k \rangle$ . Since  $s_j$  depends neither on  $\mathbf{b}^{(1)}$  nor, for  $j \neq k$ , on  $W_{\cdot k}^{(2)}$ , (12) gives  $\langle \mathbf{t}_k, \nabla_{\varphi}s_j \rangle = -s_k$  if  $j = k$  and 0 otherwise. Hence  $\langle W_{k,\cdot}^{(1)}, \mathbf{g}_k \rangle = -\nu_k s_k$ , and the first condition of (7) gives, as in the proof of Proposition 7,  $\nu_k s_k = \lambda s_k \|W_{k,\cdot}^{(1)}\|_1$ , that is  $\nu_k = \mu_k$  since  $s_k = 1$ . The stationarity condition of (C) is then  $\nabla_{\varphi}J_{\lambda} = \mathbf{0}$ . The same inner products show that  $\sum_j \nu_j \nabla_{\varphi}s_j = \mathbf{0}$  forces  $\nu = \mathbf{0}$ : the gradients of the constraints are linearly independent at the regular points of  $\Sigma$ , as claimed after the proposition.

(iii) By (6),  $s_k(G_{\mathbf{c}}\theta) W_{k,\cdot}^{(1)}(G_{\mathbf{c}}\theta) = \frac{s_k(\theta)}{c_k} c_k W_{k,\cdot}^{(1)}$ . Since  $s_k \geq 0$ ,  $s_k \|W_{k,\cdot}^{(1)}\|_1 = \|B_{k,\cdot}\|_1$ , and  $s = \mathbf{1}$  on  $\Sigma$ .

**Proof of Proposition 8.** (a)  $G_{\mathbf{c}}$  multiplies  $\mathbf{h}_i^{(1)}$  entrywise by  $\mathbf{c}$ , whose entries are positive, so that  $D_i^{(1)}$  is unchanged and, by positive homogeneity,  $\sigma_1(\mathbf{h}_i^{(1)})$  is multiplied by  $\mathbf{c}$ . The columns of  $W^{(2)}$  being divided by  $\mathbf{c}$ ,  $\mathbf{h}_i^{(2)}$  and all deeper pre-activations are unchanged. So are  $f_{\boldsymbol{\theta}}$  and the activation patterns, and, by Proposition 1,  $J_{\lambda}$ .

(b) By (6),  $s(G_{\mathbf{c}}\boldsymbol{\theta}) = s(\boldsymbol{\theta}) \odot \mathbf{c}$ , so that  $G_{s(G_{\mathbf{c}}\boldsymbol{\theta})}G_{\mathbf{c}}\boldsymbol{\theta} = G_{s(\boldsymbol{\theta}) \odot \mathbf{c}}G_{\mathbf{c}}\boldsymbol{\theta} = G_{s(\boldsymbol{\theta})}\boldsymbol{\theta}$ : both starting points are retracted onto the same representative, where the gradients coincide. The transport of the moments by  $G_{\mathbf{c}}$ , followed by their transport by  $G_{s(\boldsymbol{\theta}) \odot \mathbf{c}}$ , is their transport by  $G_{s(\boldsymbol{\theta})}$ , the factors being multiplicative. The moments therefore coincide after the first retraction as well, and the two runs carry out the same operations from then on.

(c) On  $\Sigma$ , the retraction is the identity. For scalars  $x$  and  $g$ , a threshold  $\tau \geq 0$  and  $D > 0$ ,  $x = \mathcal{S}_{D\tau}(x - Dg)$  holds if and only if  $g = -\tau \operatorname{sign} x$  when  $x \neq 0$ , and  $|g| \leq \tau$  when  $x = 0$ . Applied to (15) with  $\tau = \lambda s_k$ ,  $W^{(1)}$  is fixed if and only if the first condition of (7) holds. On  $\varphi$ , the first moment becomes  $m' = \beta_1 m + (1 - \beta_1)\nabla_{\varphi} J_{\lambda}$ , and the step is proportional to  $m'$ ; with  $m = \mathbf{0}$ , both  $\varphi$  and  $m$  are fixed if and only if  $\nabla_{\varphi} J_{\lambda} = \mathbf{0}$ . With Proposition 3(ii), this proves (c).

(d) At  $W^{(1)} = \mathbf{0}$ ,  $\mathbf{h}_i^{(1)} = \mathbf{b}^{(1)}$  for every  $i$ : all observations share their activation patterns, and  $\mathbf{a}_k(\boldsymbol{\theta}^0) = a_{0k}\mathbf{1}$  with  $|a_{0k}| = s_k(\boldsymbol{\theta}^0) = 1$  on  $\Sigma$ . As in the proof of Theorem 4,  $\mathbf{g}_{0,k} = a_{0k}X^{\top}\hat{\mathbf{r}}_0(\mathbf{y})$ , and, by the thresholding test of Section 5.4,  $W^{(1)} = \mathbf{0}$  is left unchanged if and only if  $|X_j^{\top}\hat{\mathbf{r}}_0(\mathbf{y})| \leq \lambda$  for every  $j$ , that is  $\lambda \geq \lambda_0(X, \mathbf{y})$ . On  $\varphi$ ,  $\mu_k = 0$  since  $W^{(1)} = \mathbf{0}$ , so that  $\nabla_{\varphi} J_{\lambda} = \nabla_{\varphi} \ell$ . The output  $\eta_i$  being the same function of  $\varphi$  for every  $i$ , so is  $\partial\eta_i/\partial\varphi$ , and  $\nabla_{\varphi} \ell = (\sum_i \hat{r}_{0i}) \partial\eta_1/\partial\varphi$ . Since  $\hat{\tau}$  yields the optimal constant fit,  $c \mapsto \mathcal{L}_n(c\mathbf{1}; \mathbf{y})$  is minimal at  $c = \hat{\eta}$ , where its derivative  $\sum_i \hat{r}_{0i}$  vanishes; for the Cox model,  $\mathcal{L}_n$  is invariant under  $\boldsymbol{\eta} \mapsto \boldsymbol{\eta} + c\mathbf{1}$ , so that  $\sum_i \partial\mathcal{L}_n/\partial\eta_i = 0$  identically. Hence  $\nabla_{\varphi} J_{\lambda}(\boldsymbol{\theta}^0) = \mathbf{0}$ , and  $\varphi$  is fixed.

## References

- P.-A. Absil, R. Mahony, and R. Sepulchre. *Optimization Algorithms on Matrix Manifolds*. Princeton University Press, 2008.
- Rina Foygel Barber and Emmanuel J Candès. Controlling the false discovery rate via knockoffs. *The Annals of Statistics*, 43(5):2055–2085, 2015.
- Alexandre Belloni, Victor Chernozhukov, and Lie Wang. Square-root lasso: pivotal recovery of sparse signals via conic programming. *Biometrika*, 98(4):791–806, 2011.
- Peter J. Bickel, Ya’acov Ritov, and Alexandre B. Tsybakov. Simultaneous analysis of lasso and dantzig selector. *The Annals of Statistics*, 37(4):1705–1732, 2009. doi: 10.1214/08-AOS620.
- Emmanuel Candès, Yingying Fan, Lucas Janson, and Jinchi Lv. Panning for gold: Model-X knockoffs for high-dimensional controlled variable selection. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 80(3):551–577, 2018. doi: 10.1111/rssb.12265.
- Patrick L. Combettes and B` ăng Công Vũ. Variable metric forward–backward splitting with applications to monotone inclusions in duality. *Optimization*, 63(9):1289–1318, 2014.
- Damek Davis, Dmitriy Drusvyatskiy, Sham Kakade, and Jason D. Lee. Stochastic subgradient method converges on tame functions. *Foundations of Computational Mathematics*, 20:119–154, 2020. doi: 10.1007/s10208-018-09409-5.
- D. L. Donoho and I. M. Johnstone. Ideal spatial adaptation by wavelet shrinkage. *Biometrika*, 81(3):425–455, 1994.
- Simon S. Du, Wei Hu, and Jason D. Lee. Algorithmic regularization in learning deep homogeneous models: Layers are automatically balanced. In *Advances in Neural Information Processing Systems*, volume 31, pages 382–393, 2018.
- Jean Feng and Noah Simon. Sparse-input neural networks for high-dimensional nonparametric regression and classification. 2019.
- Jerome Friedman, Trevor Hastie, and Robert Tibshirani. Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1):1–22, 2010.
- Caroline Giacobino, Sylvain Sardy, Jairo Diaz-Rodriguez, and Nick Hengartner. Quantile universal threshold. *Electronic Journal of Statistics*, 11(2):4701–4722, 2017. doi: 10.1214/17-EJS1362.
- Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. 2017.
- Ismael Lemhadri, Feng Ruan, Louis Abraham, and Robert Tibshirani. Lassonet: A neural network with feature sparsity. *Journal of Machine Learning Research*, 22(127):1–29, 2021.

- Christos Louizos, Max Welling, and Diederik P. Kingma. Learning sparse neural networks through  $l_0$  regularization. 2018. URL <https://arxiv.org/abs/1712.01312>.
- Yang Lu, Yingying Fan, Jinchi Lv, and William Stafford Noble. Deeppink: reproducible feature selection in deep neural networks. In *Advances in Neural Information Processing Systems*, volume 31, pages 8676–8686, 2018.
- Behnam Neyshabur, Ruslan Salakhutdinov, and Nathan Srebro. Path-sgd: Path-normalized optimization in deep neural networks. In *Advances in Neural Information Processing Systems (NIPS)*, volume 28, pages 2422–2430, 2015a. URL <https://papers.nips.cc/paper/5797-path-sgd-path-normalized-optimization-in-deep-neural-networks>.
- Behnam Neyshabur, Ryota Tomioka, and Nathan Srebro. Norm-based capacity control in neural networks. In Peter Grünwald, Elad Hazan, and Satyen Kale, editors, *Proceedings of the 28th Conference on Learning Theory (COLT)*, volume 40 of *Proceedings of Machine Learning Research*, pages 1376–1401. PMLR, 2015b. URL <https://proceedings.mlr.press/v40/Neyshabur15.html>.
- Sashank J. Reddi, Satyen Kale, and Sanjiv Kumar. On the convergence of Adam and beyond. In *International Conference on Learning Representations (ICLR)*, 2018.
- Tim Salimans and Durk P. Kingma. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. In *Advances in Neural Information Processing Systems*, volume 29, 2016.
- Sylvain Sardy, Maxime van Cutsem, and Sara van de Geer. The pivotal information criterion. *ArXiv*, arXiv:2603.04172, 2026.
- Pierre Stock, Benjamin Graham, Rémi Gribonval, and Hervé Jégou. Equi-normalization of neural networks. In *International Conference on Learning Representations (ICLR)*, 2019.
- Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society. Series B (Methodological)*, 58(1):267–288, 1996.
- Robert Tibshirani, Jacob Bien, Jerome Friedman, Trevor Hastie, Noah Simon, Jonathan Taylor, and Ryan J. Tibshirani. Strong rules for discarding predictors in lasso-type problems. *Journal of the Royal Statistical Society: Series B*, 74(2):245–266, 2012. doi: 10.1111/j.1467-9868.2011.01004.x.
- Maxime van Cutsem and Sylvain Sardy. Survival of the fittest cox model: Pivotal variable selection for time-to-event data. *ArXiv*, arXiv:2510.19374, 2026.
- Yutaro Yamada, Ofir Lindenbaum, Sahand Negahban, and Yuval Kluger. Feature selection using stochastic gates. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 10648–10659. PMLR, 2020.
- Liu Yang, Jifan Zhang, Joseph Shenouda, Dimitris Papailiopoulos, Kangwook Lee, and Robert D. Nowak. PathProx: A proximal gradient algorithm for weight decay regularized

deep neural networks, 2022. Short version: A Better Way to Decay: Proximal Gradient Training Algorithms for Neural Nets, NeurIPS 2022 Workshop OPT.

Jihun Yun, Aurelie C Lozano, and Eunho Yang. Adaptive proximal gradient methods for structured neural networks. *Advances in Neural Information Processing Systems*, 34: 24365–24378, 2021.

Bo Zhao, Nima Dehmamy, Robin Walters, and Rose Yu. Symmetry teleportation for accelerated optimization. In *Advances in Neural Information Processing Systems*, volume 35, 2022.